The CLI

ar011 · 2 July 2026 · Documentation

One command-line tool — *src/cli/cli.py* — drives every simulation, training run, and measurement in this project. Notebooks never import the model code; they shell out to this tool, which writes data files (metrics, rasters, population traces, weight dumps) into an output directory, and then plot those files themselves. The CLI is the engine; the plotting lives in the notebooks.

This page is the reference for that engine. It is not a tutorial — it is the dictionary you reach for when a notebook mentions *–ei-strength 0.5* and you want to know exactly what that did.

At a glance

- Three subcommands: *sim* (forward pass + metrics), *train* (surrogate-gradient BPTT), *dump-weights* (emit weight matrices).
- Every parameter is a flag. There is no global config file; a saved run's *config.json* can be inherited with *–load-config*.
- Every run writes *config.json*, *run.sh*, and *output.log* for reproducibility.
- The CLI emits data, not figures.

Quick start

Run a metrics-only forward pass of the canonical PING network:

```
uv run python src/cli/cli.py sim --model ping --ei-strength 0.5
```

Train on MNIST for 100 epochs:

```
uv run python src/cli/cli.py train --dataset mnist --epochs 100 --lr 0.0001
```

Evaluate a trained run on the test set, replaying it at a coarser timestep:

```
uv run python src/cli/cli.py sim --infer \
  --load-config runs/foo/config.json \
  --load-weights runs/foo/weights.pth \
  --dt 0.5
```

> Each subcommand has its own complete *–help*, and that per-subcommand help is authoritative. The top-level dispatcher's group summary is hand-maintained and can lag the parser — when a flag here disagrees with reality, run *sim –help* and trust that.

Commands

sim

Run one forward pass and report firing-rate metrics. This is the cheapest mode — no training, no plots — used by the test suite, the dt-stability checks in ar003, and every notebook that needs a single inference pass over a trained or fresh network.

```
uv run python src/cli/cli.py sim --model ping --dataset mnist --digit 3
```

On its own it prints metrics. The flags below make it load trained weights, evaluate a test set, emit extra data artifacts, or inject perturbations. There is no *–image* or *–video*: where those retired flags once produced panels and sweep MP4s, *sim* now emits raw data (via *–outputs*) that the calling notebook plots, and sweep videos are assembled notebook-side from many *sim* calls.

flag	default	description

–infer	off	Load trained weights and evaluate test-set accuracy; writes *metrics.json* (and *results.json*).
–load-config PATH	—	Load a saved *config.json* and inherit its model, dataset, and parameters. Explicit CLI flags override loaded values.
–load-weights PATH	—	Path to a *weights.pth* file for inference.
–max-samples INT	all	[*–infer*] Cap the evaluation set to N samples.
–outputs OUTPUT [dots.c]	—	[*–infer*] Extra artifacts from the single forward pass (*metrics.json* always written): *per_cell_rates* (per-cell E/I Hz → *per_cell_rates.npz*), *pop_traces* (per-trial population activity → *pop_traces.npz*, base signal for PSD / f_γ), *rasters* (sparse spike indices → *rasters.npz*, for cycle-level analysis).
–tau-gaba FLOAT	inherited / 9.0	[*–infer*] Override τ_GABA (ms) to replay a trained cell under specified inhibitory dynamics. Normally unset — *–load-config* inherits the trained value.
–skip-load PREFIX [dots.c]	—	[*–infer*] Drop *state_dict* keys with these prefixes before loading (e.g. *W_ei. W_ie.*) so a fresh sub-block survives — transfer-load probes (nb038).
–perturb-mode {drop, add, add_split}	—	[*–infer*] Hidden-spike perturbation inside the forward loop: *drop* (Bernoulli mask), *add*

		(Poisson Hz), *add_split* (separate E/I Poisson). The nb037 drop/add asymmetry.
–perturb-level LEVEL [dots.c]	—	[*–perturb-mode*] One value for *drop* (probability) or *add* (Hz); two values (E Hz, I Hz) for *add_split*.
–i-override-file PATH	—	[*–infer*] NPZ with a sparse per-trial I-spike stream to substitute for the inhibitory spikes each timestep — injection dual of *–outputs rasters* (nb042).
–input-file PATH	—	NPZ with *input_spikes* (T, B, N_IN) to forward instead of Poisson input — arbitrary stimulus (nb048 digit streams).
–scale-w-in / –scale-w-ei / –scale-w-ie FLOAT	1.0	[*–infer*] Multiply loaded input / E→I / I→E weights before the forward pass — inference-time coupling sweeps without retraining (nb038).
–sample-index INT	—	Raw test-set index for a snapshot, overriding *–digit / –sample*.
–n-in / –n-inh / –n-batch INT	784 / — / 64	[synthetic-spikes] Input channels, inhibitory pool size, Poisson trials averaged.
–w-ei-mean / –w-ie-mean FLOAT	from *–ei-strength*	[synthetic-spikes] Explicit W_EI / W_IE mean (std = $0.1 \cdot$ mean).
–private-w-in	off	[synthetic-spikes] Identity W_in: one input channel per E cell.

The block from *–skip-load* down is the **generic-primitive family**: small, experiment-agnostic hooks (perturb hidden spikes, inject an inhibitory stream, forward an arbitrary input file, scale a weight block at inference). Notebooks compose these instead of importing model code — that is what keeps the notebook/CLI boundary clean.

train

Surrogate-gradient BPTT training loop. Writes *weights.pth*, *metrics.json*, a per-step *metrics.jsonl*, and *test_predictions.json*.

```
uv run python src/cli/cli.py train --model ping --dataset mnist \
  --epochs 100 --lr 0.0001 --v-grad-dampen 1000
```

–epochs 0 runs the init snapshot only — useful as a probe.

flag	**default**	**description**
–lr FLOAT	0.01	Adam learning rate. Biophysical models (ping) typically need 0.0001; current-based models 0.01.
–epochs INT	0	Number of epochs. 0 = init-snapshot probe only.
–batch-size INT	64	DataLoader batch size.
–max-samples INT	all	Cap dataset to N samples for smoke tests.
–v-grad-dampen FLOAT	80.0	Dampening factor d on the COBA membrane-voltage gradient. PING needs a much larger value ($d = 1000$ in the paper) to stabilise BPTT through the E↔I loop; COBA trains at the default. Theory in ar006.
–fr-reg-upper-theta FLOAT	0 (off)	Upper-bound target spike count per neuron per trial (θ_u). Adds $s\_u \cdot \Sigma\, relu(\langle z\_i \rangle - \theta\_u)^2$ to the loss. Cramer et al. SHD RSNN: 100.
–fr-reg-upper-strength FLOAT	0	Coefficient s_u on the upper regulariser. Cramer et al.: 0.06.
–fr-reg-mode {per-neuron, population}	*per-neuron*	Pooling axis for the regulariser. *per-neuron* concentrates pressure on the highest-firing cells (Cramer recipe); *population* uses one scalar over the grand mean, distributing pressure uniformly.
–tau-gaba FLOAT	9.0 ms	Override τ_GABA. Default = *models.py*'s value (Börgers / Buzsáki-Wang range). nb041 sweeps this across {4.5 … 27} ms while training PING from scratch; the realised gamma frequency f_γ tracks 1/τ_GABA.
–frame-rate INT	10	fps for the per-epoch observation video, where one is emitted.

dump-weights

Build the network from a config and emit its weight matrices to *weights_dump.npz* — the init state, plus (with *–load-weights*) the trained state. It runs no forward pass.

```
uv run python src/cli/cli.py dump-weights \
  --load-config runs/foo/config.json \
  --load-weights runs/foo/weights.pth \
  --out-dir runs/foo/dump
```

Keys follow *W_ff_N_init / W_ff_N_trained* (feedforward, per layer N) plus the E-I blocks *W_ei / W_ie*. This is how notebooks recover the trained readout matrix (W_out = the last *W_ff*) or compare init-vs-trained loop weights (the nb049 pruning analysis) without loading the model in-process. It takes the shared option groups plus *–load-config / –load-weights*.

Shared options

These groups are attached to every subcommand. The grouping matches what each subcommand's *–help* prints.

Network

flag	default	description
–model {ping}	*ping*	Architecture. *ping* is the COBANet with E↔I coupling; with *–ei-strength 0* the inhibitory loop is silenced for a no-rhythm control.
–n-hidden INT [INT dots.c]	dataset-dependent	Hidden layer sizes. One integer = single layer; multiple stacks layers. Defaults: mnist 1024, smnist 32, shd 256.
–readout {rate, li, spike-count, mem-mean}	*rate*	Output stage. *rate* sums last-hidden spikes and projects linearly at the final timestep. *li* is a non-spiking leaky integrator per class with max-over-time — the SHD-standard readout. *spike-count* and *mem-mean* are alternative pooling rules.
–dales-law / –no-dales-law	on	Enforce Dale's law (non-negative weights) or allow signed weights. *–no-dales-law* is used for balanced-network experiments.

–ei-layers INT [INT dots.c]	all layers	1-indexed list of hidden layers that get E-I structure. PING only.
–ei-strength FLOAT	0.5	E-I coupling strength s. Sets W_EI = s and W_IE = s · ratio.
–ei-ratio FLOAT	2.0	W_IE / W_EI.
–w-in-sparsity FLOAT	0.95	Fraction of input weights zeroed at init.
–n-e INT	1024	Excitatory pool size. With *–n-i* and *–exact-k*, sweep N at fixed fan-in K to test Vreeswijk-Sompolinsky N-invariance.
–n-i INT	256	Inhibitory pool size.
–ei-sparsity FLOAT	0.0	Sparsity of the recurrent E↔I matrices: fraction of entries zeroed, survivors rescaled by 1/(1−s) to preserve expected drive. Use ≈ 1 − K/N for Brunel/ Vreeswijk sparse random connectivity.
–exact-k	off	Fixed-fan-in (exact-K) recurrent connectivity: every post cell draws exactly K = round((1−ei_sparsity)·N_pre) inputs. No effect unless *–ei-sparsity* > 0.
–dt FLOAT	0.25	Integration timestep (ms).
–t-ms FLOAT	200	Total trial duration (ms). For synthetic-step inputs, must exceed *STEP_ON_MS* (200 ms) or the stimulus window never opens.
–tau-mem FLOAT	10 ms	Membrane time constant τ_mem. Cramer et al. SHD: 20 ms.
–tau-syn FLOAT	2 ms	Synaptic (AMPA) time constant τ_syn. Cramer et al. SHD: 10 ms. Only affects COBA / PING.
–readout-tau-out FLOAT	5 ms	Output-LIF τ_out for the *spike-count* readout.

		Smaller values prevent saturation under high-rate drive at coarse dt. No-op for *rate* / *li*.
–readout-w-out-scale FLOAT	1.0	Scalar applied to the readout matrix after *build_net*, compensating for low hidden firing rate under *mem-mean* / *spike-count*. Train-mode only.
–surrogate-slope FLOAT	1.0	Fast-sigmoid surrogate-gradient slope β. Larger = narrower active window. Cramer et al. use 40 for SHD RSNNs.
–device {cpu, mps, cuda}	auto	Compute device. Auto-detected: cuda > mps > cpu.

The drive family below also lives in the Network group. It exists for the balanced-network (Brunel / van Vreeswijk-Sompolinsky) experiments and the Lyapunov chaos probe; canonical PING runs leave all of it off.

flag	**default**	**description**
–independent-drive RATE G_PER_SPIKE	off	Per-E-cell independent Poisson drive (bypasses W_in): N_E uncorrelated streams at RATE Hz, each spike adding G_PER_SPIKE µS of g_E. Zero cross-cell correlation.
–independent-drive-i RATE G_PER_SPIKE	off	As above, targeting the I population directly. Needed for the full V&S asynchronous-irregular state.
–shared-drive RATE G_PER_SPIKE	off	One Poisson stream at RATE Hz broadcast to every E cell — fully correlated. Combine with *–independent-drive* to tune cross-cell input correlation.
–shared-drive-i RATE G_PER_SPIKE	off	As *–shared-drive* but for I cells.
–noise-std FLOAT	0	Additive white-noise into each E cell's excitatory conductance every timestep — vary input SNR independently of the mean.
–quenched-drive MEAN STD	off	Per-E-cell DC conductance drawn once from N(MEAN, STD) µS and frozen for the trial — V&S quenched input. No fluctuation,

		so it cannot pin spike times; the Lyapunov probe then measures autonomous chaos.
–quenched-drive-i MEAN STD	off	Per-I-cell frozen DC conductance.
–lyapunov-eps FLOAT	0 (off)	If $>$ 0 (synthetic-spikes mode), rerun with all membranes ε-perturbed at t=0 and save the divergence $\|\Delta V(t)\|$ to *snapshot.npz*. Its growth rate is the max Lyapunov exponent: positive for the chaotic V&S state, $\approx$ 0 for cycle-locked PING.

Input

flag	**default**	**description**
–input {synthetic-conductance, synthetic-spikes, dataset}	*synthetic-spikes*	Stimulus regime. *synthetic-spikes* is Poisson at *–input-rate*. *synthetic-conductance* is a step current with *–stim-overdrive*. *dataset* draws from *–dataset*.
–input-rate FLOAT	25	Baseline input rate (Hz).
–stim-overdrive FLOAT	1.0	Stimulus multiplier.
–digit INT	0	Dataset class (0–9).
–sample INT	0	Sample index within the class.
–sample-index INT	—	Raw test-set index, overriding *–digit* / *–sample*.
–dataset {mnist, smnist, shd}	*mnist*	*mnist* = full 28×28; *smnist* = MNIST row-by-row over time; *shd* = Spiking Heidelberg Digits au-

		dio (700 channels). SHD is cached under *$PINGLAB_SHD*

Weights (advanced)

flag	default	description
–w-in MEAN [STD]	*0.3 0.06*	Input fan-in init. Single value sets STD = MEAN × 0.1.
–w-ei MEAN STD	from *–ei-strength*	Override the W_EI init.
–w-ie MEAN STD	from *–ei-strength / –ei-ratio*	Override the W_IE init.
–w-ii MEAN STD	*0 0*	W_II (I→I) init. Off by default (canonical PING has no I→I). Enable for balanced-network experiments.
–w-ee MEAN STD	*0 0*	W_EE (E→E) init. Off by default. Enable for the full four-coupling balanced network, where recurrent excitation pins the E rate.
–trainable-w-ei	frozen	Promote E→I to gradient-carrying. Asks whether the optimiser will discover the PING-loop weights from scratch.
–trainable-w-ie	frozen	Promote I→E. The nb049 result *gradient descent dismantles PING* comes from flipping *–trainable-w-ei* and *–trainable-w-ie* on simultaneously.

Output

flag	default	description
–out-dir DIR	*src/artifacts/oscilloscope/*	Output directory.
–wipe-dir	off	Clear the output directory before the run.
–raster {scatter, imshow}	*scatter*	Raster-plot style for the data emitted to raster artifacts.

Execution

flag	default	description
–seed INT	—	RNG seed. Seeds Python, NumPy, torch (CPU + CUDA + MPS) before dataset load and model init. Persisted to *config.json*.
–modal	off	Re-dispatch to Modal.com. Artifacts sync back to *–out-dir* after completion.

–modal-gpu {none, T4, L4, A10G, A100, H100}	*T4*	GPU type for Modal runs. *none* runs CPU-only.

–modal costs money. The project default is local; only pass it when explicitly instructed.

Config inheritance

The trick that makes the notebook chain work is *–load-config* (successor to the old *–from-dir*). Every *train* run writes a *config.json* alongside its *weights.pth*; a later *sim* or *dump-weights* run inherits from it:

```
uv run python src/cli/cli.py sim --infer \
  --load-config runs/foo/config.json \
  --load-weights runs/foo/weights.pth \
  --dt 0.5
```

This inherits the model, hidden sizes, dataset, E-I parameters, input rate, τ_GABA, and seed — while the explicitly-passed *–dt 0.5* overrides the trained value, replaying the network at a new timestep. Precedence is: explicit CLI flag > loaded config > default. The parser builds the set of CLI-explicit flags from *sys.argv* before applying inheritance.

Backwards compatibility: old configs that stored *n_hidden* as a scalar are remapped to the *hidden_sizes* list, legacy model names are aliased with a one-line stderr note, and configs missing *dales_law* trigger a warning to pass it explicitly or retrain.

Artifacts

Every subcommand calls *save_run_artifacts* on entry, producing three files in *–out-dir*:

file	**contents**
config.json	The parsed argparse namespace plus a provenance block (*git_sha*, *torch_version*, *device*, *python_env_hash*, *run_id*, *started_at*). Consumed by *–load-config* and the notebook metadata extractors.
run.sh	The literal *sys.argv* joined with spaces and prefixed with a shebang. Re-running reproduces the run (modulo seeded RNG state, also in *config.json*).
output.log	The run log. ANSI escapes are stripped from the file but preserved on stdout, so terminals see colour while the log stays grep-friendly.

Each command adds its own outputs: *train* writes *weights.pth*, *metrics.json*, *metrics.jsonl*, *test_predictions.json*; *sim –infer* writes *metrics.json* plus whatever *–outputs* requested; *dump-weights* writes *weights_dump.npz*.

Recipes

Train, then measure the trained network. Train writes a run directory; *sim –infer* reads it back and emits the population traces a notebook needs for a PSD:

```
uv run python src/cli/cli.py train --dataset mnist --epochs 100 \
  --lr 0.0001 --v-grad-dampen 1000 --out-dir runs/ping

uv run python src/cli/cli.py sim --infer \
  --load-config runs/ping/config.json \
  --load-weights runs/ping/weights.pth \
  --outputs pop_traces per_cell_rates
```

Loop-transfer at inference (nb038). Take a network trained as COBA and scale its E→I coupling up at inference, with no retraining:

```
uv run python src/cli/cli.py sim --infer \
  --load-config runs/coba/config.json \
  --load-weights runs/coba/weights.pth \
  --scale-w-ei 1.0 --scale-w-ie 1.0 --outputs rasters
```

Perturbation sweep (nb037). Drop a fraction of emitted spikes, or add off-phase Poisson noise, inside the forward loop:

```
uv run python src/cli/cli.py sim --infer \
  --load-config runs/ping/config.json --load-weights runs/ping/weights.pth \
  --perturb-mode drop --perturb-level 0.8
```

Recover the trained readout matrix. Dump weights and read *W_ff_N_trained* (the last layer is W_out):

```
uv run python src/cli/cli.py dump-weights \
  --load-config runs/ping/config.json \
  --load-weights runs/ping/weights.pth --out-dir runs/ping/dump
```